Unauthorised Access to CI/CD Secrets response

Supply Chain / DevSecOps · **Critical** severity · SOC IR Playbook

Incident type	Credential Exposure - CI/CD Security Breach
Severity	Critical
Priority	Critical
Detection sources	SIEM, Secret Scanning Tools, CI/CD Audit Logs, CSPM, Threat Intelligence, Bug Bounty Reports

Scenario

Secrets (such as cloud credentials, API tokens, SSH keys or environment variables) stored in CI/CD tools (e.g., Jenkins, GitHub Actions, GitLab CI, Azure DevOps) are accessed by an unauthorised party-either through misconfiguration, leaked logs, compromised runners or malicious pull requests.

MITRE ATT&CK

T1059, T1078.004, T1529, T1552.004

Preparation

Store secrets in vaults Use Secret Managers (e.g., HashiCorp Vault, AWS Secrets Manager) instead of plaintext CI/CD variables Apply least privilege Limit CI jobs and service accounts to only required permissions Monitor CI/CD audit logs Enable logging on runners, workflows and secret access Scan repositories and pipelines Use tools like TruffleHog, Gitleaks or GitHub Secret Scanning to detect exposed secrets Secure CI/CD runners Isolate, update and protect runners from tampering or privilege escalation

Detection & Analysis

Alert triggered Access or exfiltration of secrets from pipeline logs or vault Identify accessed secrets What secrets were exposed and what systems do they control? Review CI job and trigger source Determine if this was a malicious job, PR abuse or insider misuse Analyse logs and runtime metadata Inspect job logs, runner behaviour, environment variables and external callbacks

Containment

Revoke exposed secrets Immediately disable or rotate credentials, tokens and keys Disable CI jobs or pipelines Especially those that were abused or scheduled to rerun Lock down affected repositories or runners Prevent further job execution and isolate suspicious PRs or commits Notify affected platform and security teams Alert developers, DevOps and SecOps

Eradication

Delete or clean vulnerable jobs or workflows Remove embedded secrets or log outputs containing them Rebuild and secure runners Apply security updates, audit for rootkits or persistence and redeploy Tighten secret handling Use environment-level injection via secure vaults instead of hardcoded secrets Update access control lists Remove over-permissive roles or default trust to external contributors

Recovery

Rotate secrets in affected systems Cloud accounts, APIs, databases, etc. Resume CI/CD operations After full validation and hardening of build jobs, runners and configs Apply monitoring to rebuilt environments Include anomaly detection on secret use and build behaviour Restore legitimate PRs and code commits Once verified as safe and authorised

Lessons Learned & Reporting

Conduct RCA Identify root cause-vault misconfig, insider threat, leaked logs or misused permissions Update CI/CD security policies Enforce PR approval workflows, job restrictions and vault-only secrets Train DevSecOps teams

On safe secret management and pipeline hygiene Report to external stakeholders If exposed secrets impacted clients, customers or regulated data Document findings in runbook Include indicators of compromise, timelines and detection gaps

Tools typically involved

CI/CD Platforms (e.g., GitHub Actions, GitLab CI, Jenkins, Azure DevOps); Secret Management Systems (e.g., AWS Secrets Manager, Vault); Secret Scanning Tools (e.g., TruffleHog, Gitleaks, GitGuardian); SIEM (e.g., Splunk, Sentinel); SOAR (for response automation); CSPM (for cloud environment hardening and secret detection); Endpoint Monitoring (if runners or developers were targeted)

Success metrics (SLA targets)

Secret Revocation Time <15 minutes from detection CI Job Suspension Time <30 minutes Impacted Secrets Replacement Time <4 hours Secure Runner Redeployment Time <24 hours Developer Training Completion 100% of relevant team within 3 business days